

Finance Project

- 다음 거래일 방향 예측 -

Department of Data Science

Ewha Womans University

Prof. Sangyeop Kim



이화여자대학교
EWA WOMANS UNIVERSITY

팀 구성

팀 뽑기

학생 명단

엑셀 파일 불러오기

김지원
김진아
남재은
백지현
공지율

16명

팀 수

- 6 +

섞는 횟수

- 20 +

시드 - 비워 두면 시작할 때 자동으로 뽑아요

487977

랜덤 뽑기

연출 속도

섞고 배정하기

일시정지

초기화

대기 중

16명 준비됨

시드 섞기 인원

- 0/0 16

김지원	김진아	남재은	백지현	공지율	김민영
김서영	김형신	송유나	안채민	유채연	이연욱
이재연	장서연	형지원	박한서		

- API 비용: 팀당 14만원. 제조 과제에도 써야하므로 절반 정도만 사용 권장
- 후반기 제조 과제를 시작할 때 팀을 다시 구성



Task: 무엇을 예측하는가?

다음 거래일 개장 직전에 그날 종가가 전날 종가보다 오를지 내릴지를 맞히는 문제

label	구간	실제 비율
0 급하락	-2.5% 미만	7.4%
1 하락	-2.5% ~ -0.6%	24.1%
2 보합	-0.6% ~ +0.6%	32.9%
3 상승	+0.6% ~ +2.5%	27.4%
4 급상승	+2.5% 초과	8.3%

예측 시점은 대상일 09:30, 개장 직전



데이터

50개 종목, 기준일 2024-08-13 ~ 2026-09-11, 522일. 전부 합쳐 약 3.7GB

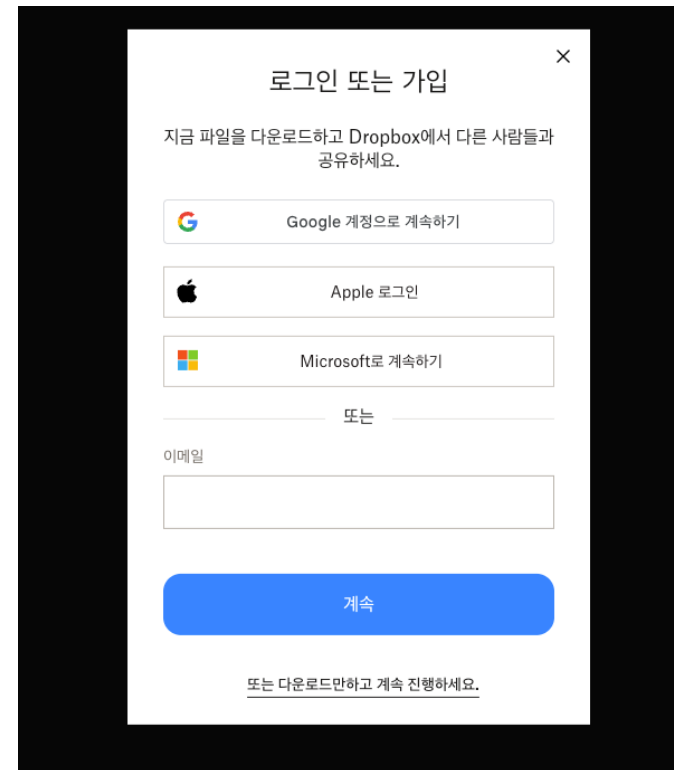
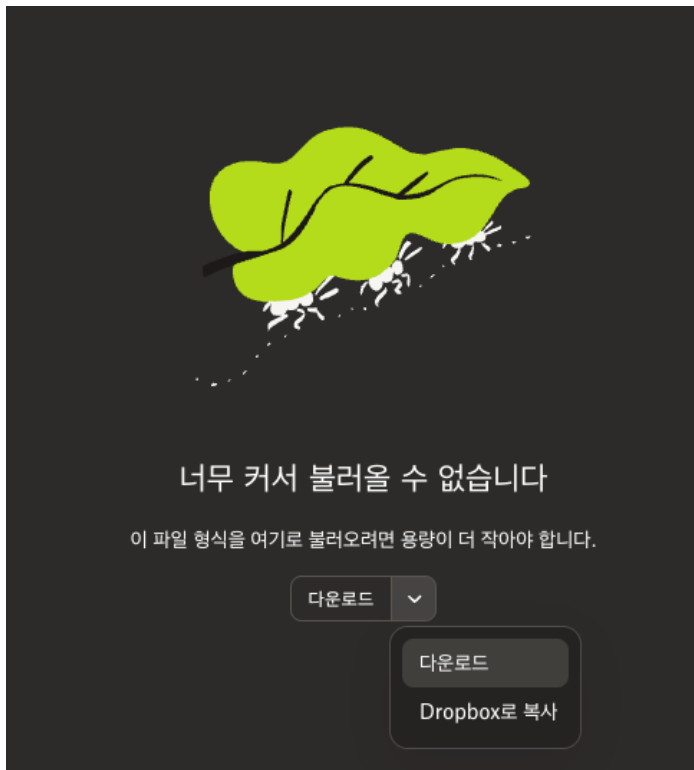
일봉과 시간봉은 2023-10부터 있어 지표 계산용

표	내용	크기
daily	일봉. 전날 종가와 등락률	36,293 행
price	1시간봉. 정규장과 시간외	590,024 행
news	기사 제목, 논조, 본문	3,203,409 행
earnings	실적 발표. EPS 예상과 실제	1,200 행
analyst	목표주가, 투자의견 변경	23,504 행
reddit	글과 댓글. subreddit 8개	3,073MB



데이터 받기

<https://www.dropbox.com/scl/fi/v7nur92m5kp8fku9txn5c/dataset.tar.gz?rlkey=wrg56c2apbz9x0285xtowvj08>



로그인 창이 뜨면 아래의 ‘또는 다운로드만하고 계속 진행하세요.’를 클릭



소스코드 받기

<https://github.com/Ewha-DAIA/Finance>

 **Finance** Public Edit Pins Watch

main 1 Branch 0 Tags Add file Code

 **sangyeop-kim** first

72f3a38 · 1 minute ago 1 Commit

example	first	1 minute ago
src	first	1 minute ago
.gitignore	first	1 minute ago
README.md	first	1 minute ago
main.py	first	1 minute ago
pyproject.toml	first	1 minute ago

README

주가 방향 예측

```
git clone https://github.com/Ewha-DAIA/Finance.git
```



known_at

모든 표에 있는 known_at은 그 값을 언제부터 알 수 있었는지를 담은 열

	일어난 시각	알 수 있었던 시각
1시간봉	봉의 시작 09:30	봉이 닫히는 10:30
일봉	그날	16:00 마감
실적	발표 시각	같음
Reddit 글·댓글	올라온 때	올라온 직후. created_et 로 자름
Reddit score, n_comments	글이 올라온 때	36시간 뒤. feature 로 쓰지 말 것

자르는 기준은 known_at 하나뿐. Reddit만 known_at이 없어 올라온 시각(created_et)으로 자름



데이터 불러오기

```
ds = Dataset()  train, test = ds.split("2026-06-01")  day = test[0]
```

cutoff 까지 들어옴 : 기준일 장 마감 후 뉴스, 대상일 개장 전 시간외 거래

호출	돌려주는 것
day.date / day.target / day.cutoff	기준일, 예측 대상일, 대상일 09:30
day.daily(days=20)	최근 20거래일 일봉
day.price(days=1)	시간봉
day.news(["AMD"], days=5)	기사
day.reddit("stocks", "comments")	글과 댓글

days는 거래일 기준. 구간을 아무리 넓혀도 끝은 항상 cutoff



기간 나누기

기준일 앞은 학습 구간, 기준일부터는 평가 구간

방식	코드
한 번 나누기	<code>train, test = ds.split("2026-06-01")</code>
walk-forward	기준일을 여러 개 두고 같은 함수를 반복
rolling	<code>train = train[-120:]</code>

무작위로 섞으면 내일 데이터로 오늘을 학습하게 됨



점수

$$score = 1 - \Sigma w \cdot O / \Sigma w \cdot E \quad w[i][j] = (i - j)^2 \times (|i - 2|)^2$$

i : 정답 등급, j : 예측 등급, O : 실제 혼동행렬, E : 무작위로 예측했을 때의 기대 혼동행렬

$\Sigma w \cdot O$: 아래 표의 칸마다 가중치 $\times$ 그 칸에 들어간 예측 개수를 모두 더한 값

정답 \ 예측	급하락	하락	보합	상승	급상승
급하락	0	4	16	36	64
하락	1	0	1	4	9
보합	0	0	0	0	0
상승	9	4	1	0	1
급상승	64	36	16	4	0

우연으로 맞은 몫을 빼므로 0점이 기준선, 전부 맞으면 1점



실제 혼동행렬 O

예측 10건의 예. $O[i][j]$ = 정답이 i, 예측이 j인 건수

$\sum w \cdot O$ = 칸마다 앞 쪽의 $w[i][j]$ 와 $O[i][j]$ 를 곱해 모두 더한 값

정답 \ 예측	급하락	하락
급하락	0	0
하락	0	0
보합	0	0
상승	0	0
급상승	0	0

별점이 붙는 칸은 둘. $2 \times 4 + 1 \times 16 = 24 \rightarrow \sum w \cdot O = 24$



기대 혼동행렬 E

$E[i][j] = (i\text{행 합}) \times (j\text{열 합}) / N$. 두 분포는 그대로 두고 정답과 예측의 대응만 끊음

행 합 $n = 0, 2, 5, 0, 3$ 열 합 $m = 0, 0, 6, 2, 2$ $N = 10$

$$\Sigma w \cdot E = 1.2 \times 1 + 0.4 \times 4 + 0.4 \times 9 + 1.8 \times 16 + 0.6 \times 4 = 37.6$$

정답 \ 예측	급하락	하락
급하락	0	0
하락	0	0
보합	0	0
상승	0	0
급상승	0	0

$$\text{score} = 1 - 24 / 37.6 = 0.36$$



어떤 예측이 높은 점수를 받는가

전부 보합은 정확도 32.9%인데 점수는 0. 급등락만 잡는 쪽은 정확도 48.6%로 점수가 더 높음

전략	score
전부 보합, 또는 한 등급만 계속	0.000
방향만 맞고 세기는 틀림	+0.827
급등락만 잡고 나머지 보합	+0.896
급등락을 정반대로	-1.781
전부 맞힘	+1.000

급등락을 반대로 찍으면 -1 아래로 내려감. 가중이 비대칭이라 하한이 없음



제출과 채점

src/model.py에 load_model()과 Model.predict(day) 두 가지만 있으면 됨

항목	규칙
제출물	소스코드 (src/model.py 와 artifacts/. 나머지 src/ 는 채점자가 원본으로 덮어씀)
출력	symbol 과 label 두 열. label 은 0부터 4까지의 정수
학습 종목	day.symbols 에 있는 종목만
평가 구간	공개하지 않은 이후 기간 (10월, 9월 중순부터 말까지는 embargo)
평가 종목	공개한 50개 중 10개 종목과 새로운 10개 종목

- predict --date 는 마지막 거래일(09-14)에는 쓸 수 없음. 다음 거래일 정답이 없기 때문

채점할 때 학습을 다시 돌리지 않음. 제출 전 python main.py check로 확인



평가

성능은 100점 중 30점. 나머지 70점은 방법, 설명, 발표, 코드에 배분

항목	배점	무엇을 보는가
향후 성능 평가	30점	공개하지 않은 구간에서의 score
정성 평가	30점	방법의 참신성, 논리, 수업 내용의 활용, 해석의 타당성
발표	15점	자료 준비 및 발표
코드	15점	재현 가능성과 구조
피어리뷰	10점	다른 팀을 평가한 참여도와 타당성

- 팀원끼리 서로를 평가한 결과는 최종 점수에 퍼센트로 반영
- 제출이 늦으면 30분마다 3%p 감점. 30분 이내 3%p, 한 시간 이내 6%p



환경 설정: uv

패키지 설치, 가상환경, 버전 고정을 한 도구로 처리

항목	uv	pip	conda
속도	Rust로 작성, 10~100배	느림	매우 느림
의존성 해결	전체 트리	부분	전체, 느림
버전 고정 파일	uv.lock	없음	environment.yml
가상환경	내장	venv 별도	conda env
Python 버전	uv python install	없음	있음

uv.lock이 있으면 어느 컴퓨터에서든 같은 버전이 설치됨



의존성과 uv.lock

패키지 하나를 설치하면 그 패키지가 요구하는 패키지들도 함께 설치됨

항목	내용
문제	A는 B 2.0 이상, C는 B 2.5 미만을 요구. 둘을 동시에 만족해야 함
uv의 처리	모든 제약을 한 번에 읽고 조건을 만족하는 버전 조합을 탐색
pyproject.toml	직접 설치한 패키지와 필요한 최소 버전
uv.lock	함께 설치된 것까지 포함한 모든 패키지의 정확한 버전

uv.lock 은 uv sync 가 자동으로 만듦. 직접 고치지 말 것. 팀 저장소에 함께 올리면 팀원 간 버전이 맞음



uv 설치와 환경 준비

macOS, Linux : `curl -Ls https://astral.sh/uv/install.sh | sh`

Windows : `powershell -c "irm https://astral.sh/uv/install.ps1 | iex"`

명령	하는 일
<code>uv sync</code>	<code>uv.lock</code> 에 적힌 버전 그대로 전부 설치
<code>uv add <pkg></code>	패키지 설치 후 <code>pyproject.toml</code> 과 <code>uv.lock</code> 갱신
<code>uv run python main.py</code>	프로젝트 환경 안에서 실행
<code>uv python install 3.12</code>	지정한 Python 버전을 내려받아 설치

- 설치 안내 <https://wikidocs.net/305829>

